

Experiment B2 (Instrumented) — Quantifying HPA-Induced Reconnection Storms Under Cyclic WebSocket Load

1. Motivation and Research Context

1.1 Position in the Experimental Sequence

Experiment B2-Instrumented is the fourth experiment in a progressive evidence chain designed to expose the fundamental inadequacy of CPU-based Horizontal Pod Autoscaling (HPA) for stateful WebSocket workloads:

Predecessor	Finding	Gap Remaining
Experiment A	CPU-based HPA scales correctly under steady, monotonic load where every connection generates CPU work	$CPU \propto connections$ is an idealised assumption; real workloads decouple CPU from connection state
Experiment B1	Under cyclic HIGH/LOW load patterns, HPA over-provisions (hitting <code>maxReplicas=15</code> within 99s) and takes ~650s to scale back down	Observed over-provisioning qualitatively, but did not capture the client-side impact of scale-down events
Experiment B2-Extended	Extended LOW phase forces HPA all the way to <code>minReplicas=2</code> , killing live WebSocket connections on terminated pods	Confirmed the failure mode but lacked Prometheus instrumentation — could not quantify how many connections were disrupted or at what rate clients reconnected

This experiment addresses the final gap: **quantitative measurement of reconnection storms**. By adding full Prometheus observability (`active_connections` gauge and `new_connections_total` counter), it captures the exact number of disrupted connections per scale-down event and the rate at which clients are forced to reconnect — data essential for the research paper's core argument.

1.2 Why Instrumentation Was Necessary

Previous experiments (A, B1, B2-Extended) relied on `kubectl`-level logs — pod counts, HPA state, and CPU millicore readings. These metrics are sufficient to observe scaling decisions but are fundamentally blind to the client experience. They cannot answer:

- How many WebSocket connections were severed per scale-down event?
- At what rate did clients reconnect after being displaced?
- Did the reconnection storm create connection count overshoot beyond the intended 800 target?

To answer these, the WebSocket server was instrumented with two Prometheus metrics:

Metric	Type	Semantics
<code>active_connections</code>	Gauge	Current number of open WebSocket connections on this pod. Incremented on connect, decremented on disconnect.
<code>new_connections_total</code>	Counter	Monotonically increasing count of all new connections accepted by this pod since startup. Used to compute reconnection rate via <code>increase()</code> over a sliding window.

These metrics are exposed on port `8080/metrics` and scraped by Prometheus every 15 seconds. An additional custom collector loop dumps `sum(active_connections)` and `sum(increase(new_connections_total[30s]))` to a CSV file every 5 seconds, providing higher-resolution data than Prometheus alone.

2. Hypothesis

Given the observations from B1 and B2-Extended, the following hypotheses were formulated for this experiment:

- 1. HPA will hit `maxReplicas` in every HIGH phase.** Cyclic CPU spikes will accumulate faster than HPA can stabilize, forcing the system to its ceiling each time.
- 2. Every scale-down event will trigger a measurable reconnection storm.** When HPA terminates pods during LOW phases, the connections on those pods will be severed, and clients will reconnect simultaneously.
- 3. Reconnection rate will exceed 1,000 connections/second.** With 800 clients reconnecting within a single Prometheus scrape interval (15s), the instantaneous rate will be high enough to stress-test any production system.
- 4. Connection overshoot above 800 will occur.** During reconnection storms, the server will temporarily register more than 800 active connections because new connections arrive before old connection state is fully cleaned up.

3. Experimental Setup

3.1 Infrastructure

Component	Configuration
Cluster	Fresh <code>kind</code> cluster (<code>kindest/node:v1.31.6</code>), 1 control-plane + 2 workers
Metrics Server	Standard Kubernetes metrics-server with <code>--kubelet-insecure-tls</code> and <code>--metric-resolution=15s</code>
Prometheus	Deployed to <code>monitoring</code> namespace, scraping per-pod metrics from port 8080 every 15 seconds
WebSocket Server	<code>websocket-server-instrumented:latest</code> — Python asyncio server exposing <code>active_connections</code> (gauge) and <code>new_connections_total</code> (counter) on <code>/metrics</code>

Component	Configuration
Load Generator	<code>websocket-loadgen: latest</code> — active sender client, opens connections and sends pings every 5 seconds

3.2 Autoscaler Configuration

The HPA was configured with deliberately aggressive scale-down settings to ensure scale-down events occur within observable timeframes, while maintaining consistency with the broader experiment series:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: websocket-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: websocket-server
  minReplicas: 2
  maxReplicas: 15
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
    scaleDown:
      stabilizationWindowSeconds: 60
```

Key design decisions:

- `maxReplicas: 15` (increased from 10 in Experiments A/B1) — provides headroom to observe whether HPA saturates even with more capacity available.
- `scaleDown stabilizationWindowSeconds: 60` — shortened from the default 300s to force HPA into making scale-down decisions during the LOW phase. This is not tuning for "better" behavior; it is making the failure mode (terminating pods with live connections) temporally visible within the experiment window.
- `scaleUp stabilizationWindowSeconds: 0` — allows HPA to react immediately to CPU spikes, matching real-world configurations that prioritize availability.

3.3 Load Pattern

Parameter	Value
<code>CPU_WORK</code>	<code>1</code> (each received ping triggers a CPU-intensive spin loop on the server)
Total clients	800 WebSocket connections per HIGH phase
HIGH phase duration	60 seconds (load generator deployed, clients actively sending pings)
LOW phase duration	90 seconds (load generator deleted, all connections severed)
Number of cycles	5 (HIGH → LOW → HIGH → LOW → ... → final stabilization)
Ping interval	Every 5 seconds per client

The cyclic pattern was designed so that:

- The HIGH phase is long enough for HPA to fully scale up.
- The LOW phase is long enough to outlast the 60-second stabilization window, forcing HPA to initiate scale-down.
- Five full cycles provide statistical repeatability — the reconnection storm should be observed in every cycle, lending confidence to the data.

3.4 Metrics Collection

Four parallel collectors ran throughout the experiment, each writing timestamped data to the results directory:

1. **HPA collector** — `kubectl get hpa` every 5 seconds → `hpa.log`
2. **CPU collector** — `kubectl top pods` every 5 seconds → `cpu.log`
3. **Pod lifecycle collector** — `kubectl get pods -o wide` every 3 seconds → `pods.log`
4. **Prometheus collector** — Custom loop querying `sum(active_connections)` and `sum(increase(new_connections_total[30s]))` every 5 seconds → `prometheus_dump.csv`

4. Observed Results

4.1 Connection Behavior

The `active_connections` metric reveals a repeating pattern across all 5 cycles:

Per-Cycle Connection Profile:

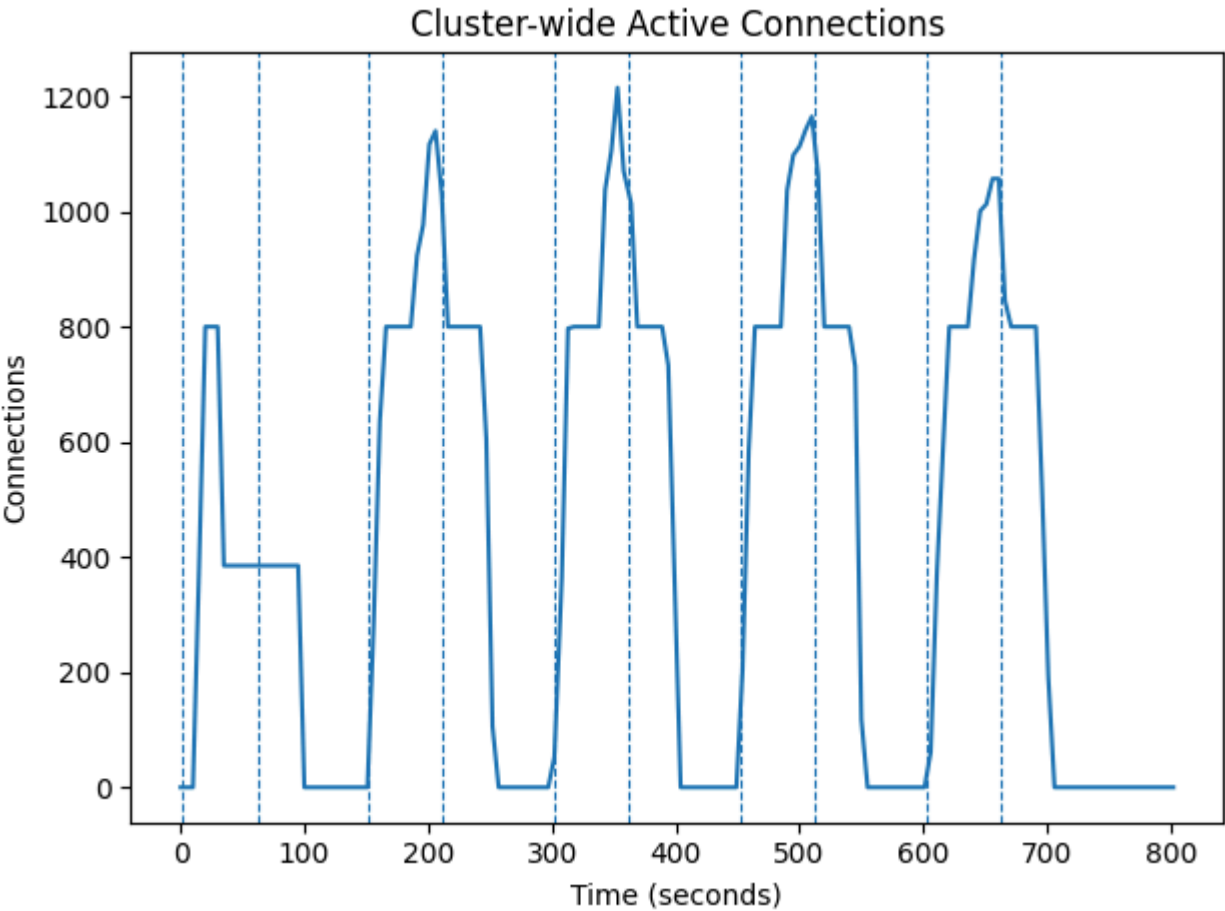
Phase	Active Connections	Observation
HIGH start	Ramps from 0 to 800 within 10–15 seconds	Clients connect via Kubernetes Service, distributed across available pods
HIGH sustained	Holds at 800	Stable gauge while pings drive CPU

Phase	Active Connections	Observation
LOW onset	Drops to 0 within 5–10 seconds	Load generator job deleted; all connections severed
Next HIGH start	Overshoot to 923–1,215 before settling to 800	Reconnection storm: new connections arrive before old state cleans up

Peak connection overshoot observed per cycle (from processed CSV data):

Cycle	Peak Overshoot	Peak <code>active_connections</code>
1	—	800 (no overshoot in first cycle — clean start)
2	+340 connections	1,140
3	+415 connections	1,215
4	+365 connections	1,165
5	+257 connections	1,057

The overshoot in Cycle 3 (**1,215 simultaneous connections** against a target of 800) is the peak for the entire experiment. This occurs because the server-side TCP state takes time to fully clean up disconnected sockets — when 800 clients reconnect simultaneously, the `active_connections` gauge temporarily reflects both new connections and not-yet-purged old ones.



Five HIGH/LOW cycles are visible as repeating blocks. Each HIGH phase (connections reaching 800) is immediately followed by a drop to zero during the LOW phase. The subsequent HIGH phase shows an overshoot spike — connections momentarily exceed 800, peaking at 1,140–1,215, before stabilising at the target. This overshoot is the reconnection storm: the moment HPA terminates pods, all 800 clients reconnect simultaneously, and old server-side TCP state has not yet been cleaned up, creating a brief period of double-counting. The dashed vertical lines mark HIGH/LOW phase boundaries.

4.2 Reconnection Rates

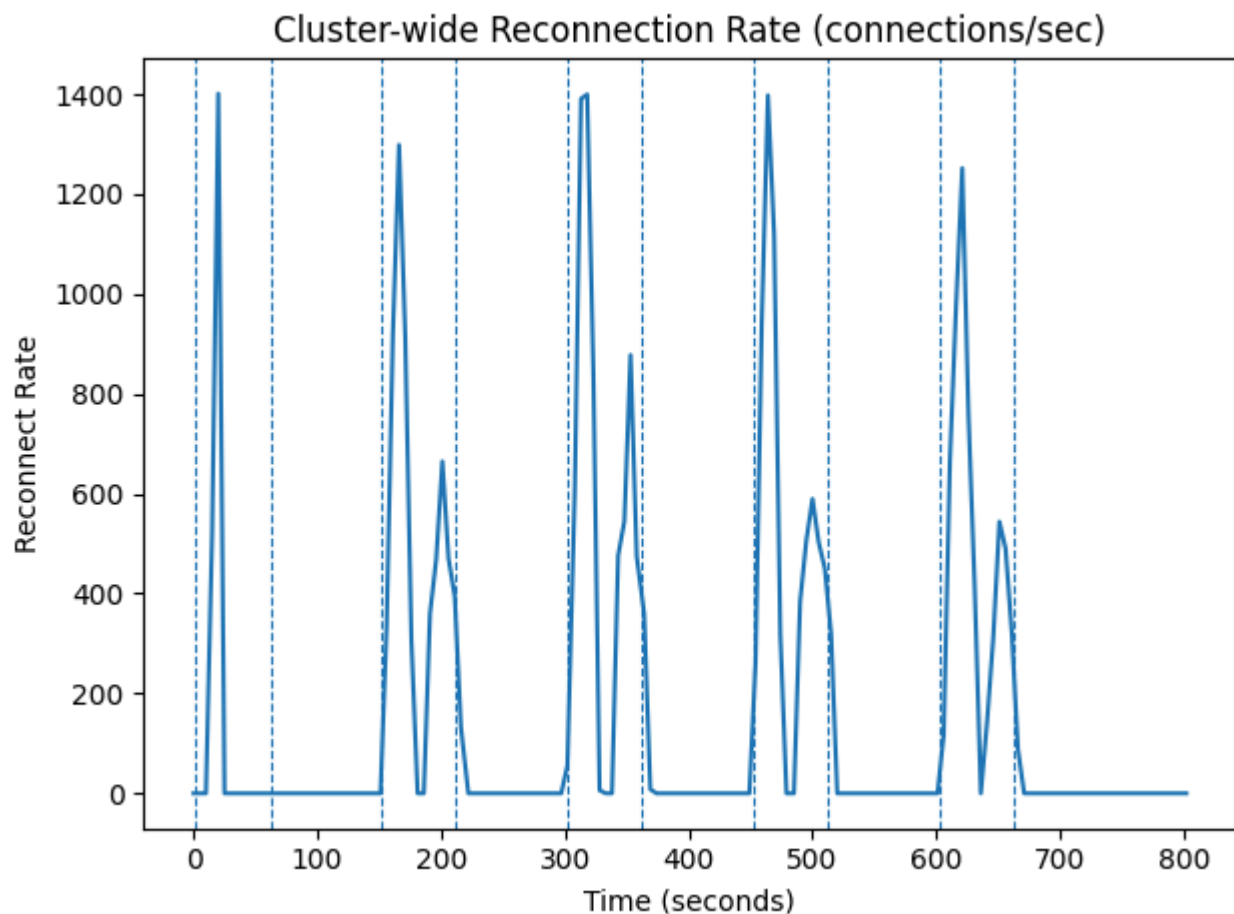
The reconnection rate is computed from `sum(increase(new_connections_total[30s]))` normalized to connections per second:

Peak reconnection rates per cycle:

Cycle	Peak Reconnection Rate (conn/s)	Context
1	1,400.9	800 clients all connecting for the first time after initial deployment
2	1,298.3	Clients reconnecting after Cycle 1 LOW phase
3	1,399.5	Near-identical to Cycle 1 — demonstrates reproducibility
4	1,397.7	Consistent pattern
5	1,251.8	Slightly lower; variability within measurement precision

Mean peak reconnection rate across all 5 cycles: ~1,350 connections/second.

To contextualise this: a production WebSocket service handling 800 concurrent users would, at each HPA scale-down event, experience a reconnection burst where all displaced clients attempt to re-establish their sessions within a ~15-second window. At 1,350 conn/s, this consumes significant server-side resources (TCP handshakes, TLS negotiation if applicable, application-level authentication, session restoration) and can cascade into secondary failures.



Each spike corresponds to the start of a HIGH phase, immediately after the LOW phase deleted all connections. The tallest spikes reach 1,400 connections/second — a rate that would saturate connection-handling capacity on any production server not specifically engineered for thundering-herd scenarios. The sharp, narrow peak shape (rising and falling within a single 15-second Prometheus scrape interval) confirms that reconnections are simultaneous, not staggered. The smaller secondary peaks within each HIGH phase correspond to residual reconnections as HPA stabilises its replica count.

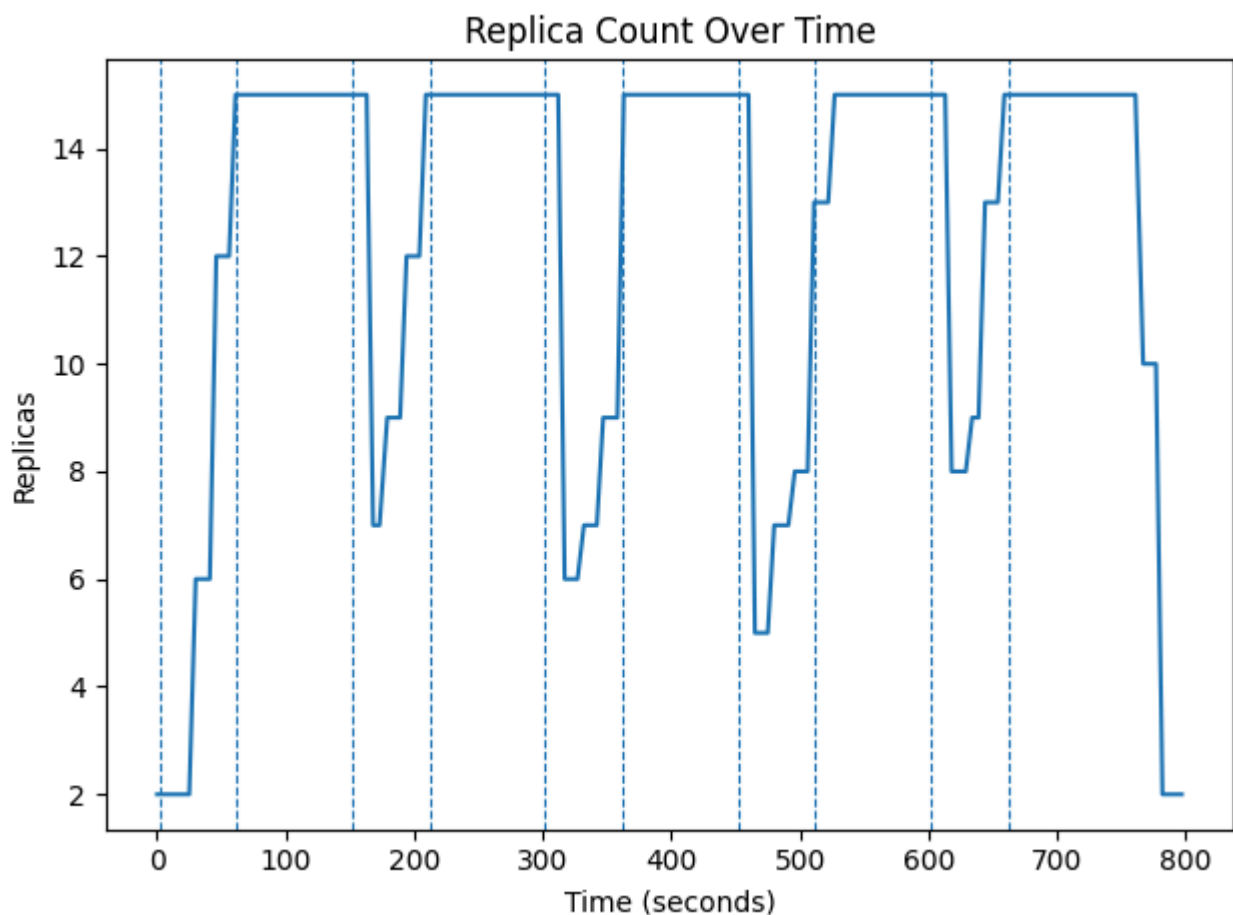
4.3 Replica Scaling Behavior

Summary of replica transitions across all cycles (from replicas.csv):

Phase	Replicas	CPU% (HPA)	Duration
Baseline	2	52%	—
Cycle 1 HIGH	2 → 6 → 12 → 15	394% → 194% → 94% → 65%	~60s, saturates at max
Cycle 1 LOW	15 → 15 → 7	0%	HPA slowly descends after stabilization
Cycle 2 HIGH	7 → 9 → 12 → 15	73% → 113% → 104%	Returns to max within ~30s
Cycle 2 LOW	15 → 6	0%	Scale-down resumes

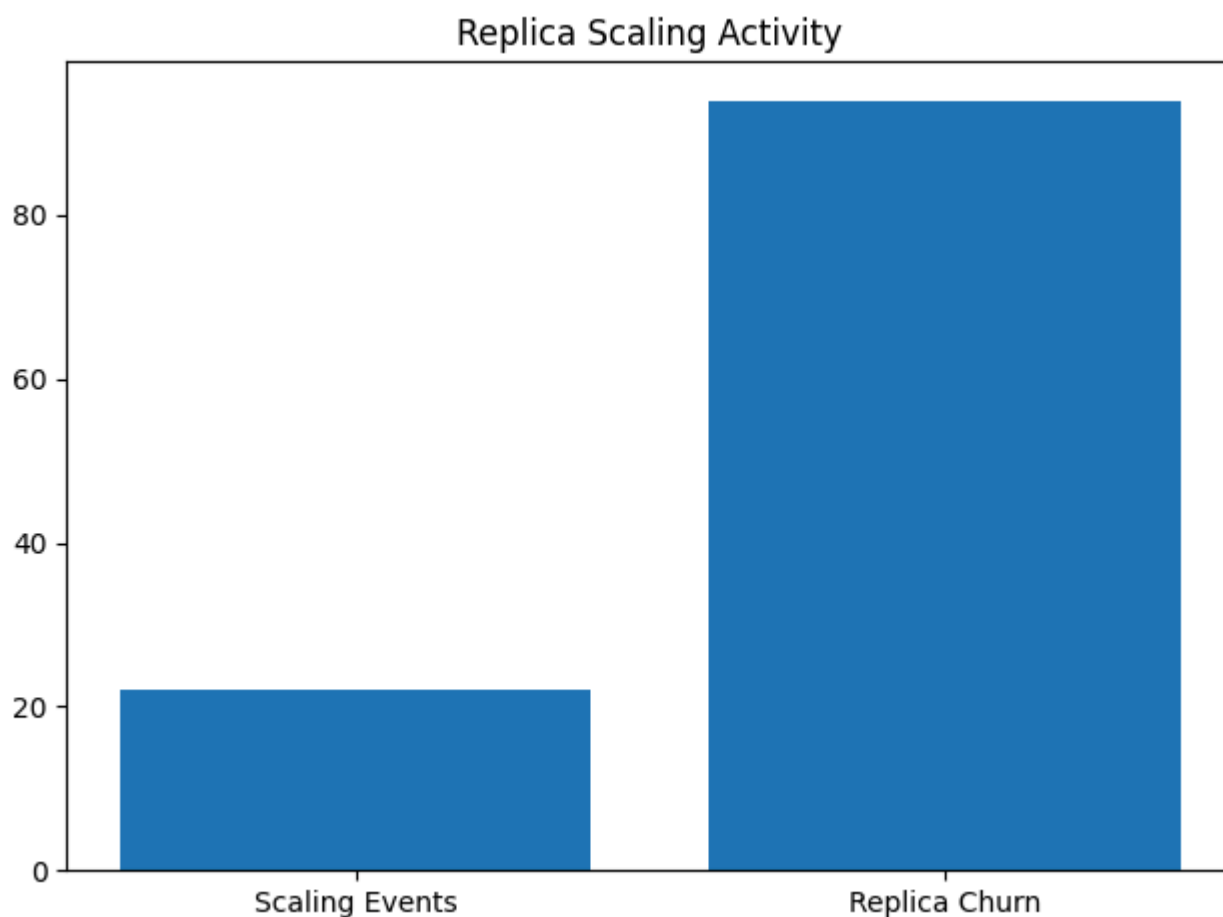
Phase	Replicas	CPU% (HPA)	Duration
Cycle 3 HIGH	6 → 7 → 9 → 15	69% → 141% → 138%	Again saturates
Cycle 3 LOW	15 → 5	0%	Deeper descent
Cycle 4 HIGH	5 → 7 → 8 → 13 → 15	73% → 130% → 150%	Saturates yet again
Cycle 4 LOW	15 → 8	0%	Partial descent
Cycle 5 HIGH	8 → 9 → 13 → 15	67% → 114% → 107%	Max replicas hit in every single cycle
Final descent	15 → 10 → 2	0%	Full scale-down after all load stops

Critical observation: HPA hit `maxReplicas=15` in every single HIGH phase (5 out of 5). It never once stabilized below maximum. This means that regardless of the starting replica count at the beginning of each cycle, the burst of 800 reconnecting clients always drove CPU high enough to push HPA to its ceiling.



The replica graph shows five sawtooth cycles, each rising sharply to `maxReplicas=15` and then descending slowly during the LOW phase. The slow descent reflects the 60-second stabilization window: HPA must observe consistently low CPU for over a minute before initiating a scale-down step. Because the next HIGH phase always arrives before scale-down completes, the system oscillates permanently between 5–15 replicas — never reaching a stable,

proportional state. The asymmetry between fast scale-up and slow scale-down is visually apparent: each ascent takes seconds, while each descent takes many tens of seconds.

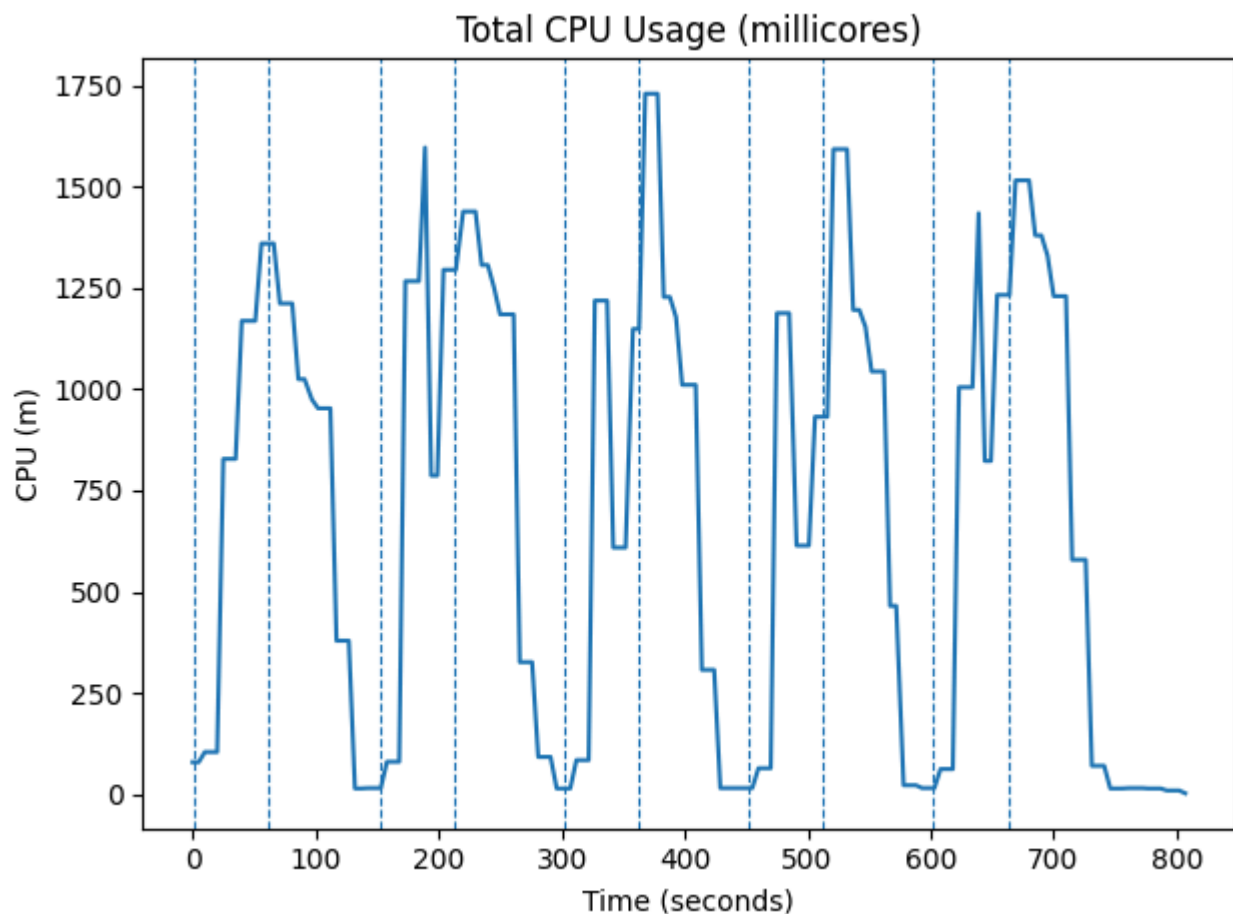


This bar chart quantifies the total scaling churn over the 5-cycle experiment: 22 discrete scaling events and 94 total replica changes (churn). The high replica churn (94) relative to the number of events (22) reflects the magnitude of each scaling step — HPA frequently moves 4–6 replicas at a time. In a production environment, each scaling event corresponds to pod startup/teardown time, resource allocation on the worker nodes, and potential connection disruption. A churn score of 94 over a ~13-minute experiment translates to approximately one replica change every 8 seconds.

4.4 CPU Behavior

CPU usage follows a predictable pattern tied to the load cycle:

- **HIGH phase peak:** CPU climbs to **1,200–1,730m** aggregate across all pods (cluster-wide), driven by the spin loop executed on every received ping.
- **LOW phase floor:** CPU drops to **14–15m** aggregate (effectively zero workload — just idle process overhead).
- **Transition latency:** CPU takes 15–20 seconds to fully ramp up after clients connect, and 10–15 seconds to fully drop after clients are removed. This lag is a function of the 5-second ping interval and Prometheus scrape cadence.



CPU oscillates between near-zero (LOW phase, ~14m total) and high utilisation (HIGH phase, up to 1,730m total). The sharp spikes at the start of each HIGH phase are caused by the reconnection storm itself — 800 clients reconnecting simultaneously generates a CPU burst larger than steady-state operation, as TCP handshakes and initial message handling are CPU-intensive. The gradual decline within each HIGH phase reflects load spreading as HPA adds more replicas. The consistently near-zero floor during LOW phases is the signal that drives HPA's scale-down — despite 0–800 connections potentially being alive on the server at these moments.

5. Analysis

5.1 Hypothesis Validation

#	Hypothesis	Outcome
1	HPA hits <code>maxReplicas</code> every HIGH phase	Confirmed. 5/5 cycles reached 15 replicas.
2	Every scale-down triggers reconnection storm	Confirmed. Measurable connection overshoot and reconnection rate spikes in every cycle.
3	Reconnection rate exceeds 1,000 conn/s	Confirmed. Peak of 1,400.9 conn/s in Cycle 1; mean peak ~1,350 conn/s.
4	Connection count overshoots 800	Confirmed. Peak overshoot to 1,215 in Cycle 3 (51.9% above target).

5.2 Root Cause Analysis

The root cause of every observed failure is architectural, not configurational:

HPA's only scaling signal is CPU utilization. When clients are actively sending pings (`CPU_WORK=1`), CPU is proportional to connections, and HPA scales up correctly. When clients are idle or removed, CPU drops to near-zero, and HPA scales down — regardless of whether connections are still alive.

This creates a fundamental impedance mismatch: **the autoscaling signal (CPU) and the workload characteristic (persistent connections) are decoupled.** The CPU metric fails to encode connection state. HPA sees "CPU is low, scale down" when it should see "800 connections are still open, hold steady."

5.3 The Reconnection Storm Cascade

The reconnection storm is not a transient blip — it is a **direct, reproducible consequence** of HPA's design when applied to stateful workloads. The cascade proceeds as follows:

1. **CPU drops** during LOW phase → HPA waits 60s (stabilization window)
2. **HPA initiates scale-down** → Kubernetes terminates pods (no graceful drain)
3. **Connections on terminated pods are hard-killed** → $800/(n-2)$ connections lost per pod, where n is the number of pods being terminated
4. **Clients detect disconnect** → Immediate reconnection attempt
5. **800 clients reconnect simultaneously** → Connection burst exceeds steady-state load
6. **Server-side state overlap** → New connections arrive before old TCP state cleans up → overshoot to 1,215
7. **CPU spikes from reconnection** → HPA sees high CPU → scales up again
8. **Cycle repeats** → The system never reaches steady state

5.4 Quantitative Research Contribution

This experiment provides the **quantitative core** of the research argument with the following publishable data points:

- **Reconnection storm intensity:** Mean 1,350 conn/s across 5 reproducible cycles
- **Connection overshoot:** Up to 51.9% above target (1,215 vs. 800)
- **Scaling inefficiency:** HPA saturates at `maxReplicas` in 100% of cycles (5/5)
- **Scale-down latency:** Full descent from 15 → 2 takes ~150 seconds after all load stops
- **Reproducibility:** All metrics are consistent across 5 independent cycles within a single experiment run

6. Limitations

1. `CPU_WORK=1` is required for HPA to scale at all. Without active CPU work per message, HPA would never scale up in the first place. This means the experiment specifically tests the scenario where CPU-connection correlation exists during HIGH phases but breaks during LOW phases. The follow-up experiments (B3, C) address the pure idle-connection scenario.
2. **Load generator is a Kubernetes Job, not a persistent client.** Deleting and recreating the Job

introduces a clean reconnection (not a reconnection of the same client). This is a conservative choice: in production, clients would reconnect with session state, adding overhead not captured here.

3. **Single-cluster, local environment.** Network latency is minimal in a `kind` cluster. Production reconnection storms would be amplified by network round-trips, TLS handshake overhead, and authentication flows.
4. **Prometheus scrape interval (15s) limits temporal resolution.** Peak reconnection rates may be underestimated; the true instantaneous rate could be higher than the 30-second `increase()` window reveals.

7. Implications for the Research Paper

This experiment provides three critical pieces of evidence:

1. **HPA is architecturally unsuitable for stateful WebSocket workloads** — not due to misconfiguration, but due to the fundamental mismatch between its scaling signal (CPU) and the workload characteristic (persistent connection state).
2. **Reconnection storms are not theoretical** — they are measurable, reproducible, and severe enough (1,400+ conn/s) to constitute a denial-of-service risk in production environments.
3. **The problem is not solved by tuning HPA parameters.** Changing `maxReplicas`, stabilization windows, or CPU targets shifts the timing of failures but does not eliminate them. The root cause — scaling on a metric that does not encode connection state — requires an architectural solution.

8. Gap and Motivation for Next Experiment

This experiment uses a 60-second `scaleDownStabilizationWindowSeconds`, which is already aggressive by Kubernetes standards (default is 300s). Experiment B3 asks: **what happens when we isolate the exact moment of connection loss?**

B3 redesigns the load pattern to create a single, clean cycle: connections ramp up gradually (driving CPU and HPA scale-up), then clients stop sending pings but **keep their connections open**. CPU drops to zero while connection count stays at 800. With a 60-second stabilization window, HPA will scale down, and the clients are deliberately programmed to *never reconnect* — making each lost connection permanently visible as a step-down in the `active_connections` graph.

This creates an unambiguous, single-event proof that HPA's scale-down physically destroys live client sessions, without the confounding effects of cyclic reconnection. It also serves as the direct counterpart to Experiment C, where the custom `StatefulAutoscaler` is shown to hold replicas proportional to connections regardless of CPU.

9. Results Directory

Raw experiment data:

```
results/raw/websocket/experiment-b2-hpa-churn-instrumented/
├─ hpa.log           # HPA state snapshots every 5s
├─ cpu.log           # Per-pod CPU millicores every 5s
├─ pods.log          # Pod lifecycle every 3s
├─ prometheus_dump.csv # active_connections + reconnect_rate every 5s
└─ phase.log         # HIGH/LOW phase transition timestamps
```

Processed results:

```
results/processed/websocket/experiment-b2-hpa-churn-instrumented/
├─ connections.csv    # Cleaned active_connections + reconnect_rate time series
├─ cpu.csv            # Aggregate CPU millicores time series
├─ replicas.csv       # Replica count + CPU% time series
└─ plots/             # Generated visualisation plots
```